JARVIS MARK-I — Day 1 Code Explanation

This document explains the Day 1 JARVIS MARK-I voice assistant code step by step. The project uses Python, Edge TTS, and Pygame to generate and play AI voice responses.

Complete Code

```
import asyncio
import edge_tts
import pygame
import os

async def speak(text):

    print(f"\n[JARVIS]: {text}")

    communicate = edge_tts.Communicate(
        text=text,
        voice="en-US-GuyNeural"
    )

    await communicate.save("voice.mp3")

    pygame.mixer.init()
    pygame.mixer.music.load("voice.mp3")
    pygame.mixer.music.play()

    while pygame.mixer.music.get_busy():
        continue

    pygame.mixer.quit()

    os.remove("voice.mp3")

print("=" * 50)
print("      JARVIS MARK-I INITIALIZING")
print("=" * 50)

asyncio.run(
    speak(
        "System online. Hello Divyansh."
    )
)
```

1. import asyncio

The asyncio module helps run asynchronous functions. It allows the program to wait for tasks like generating AI speech without freezing the application.

2. import edge_tts

edge_tts is the AI voice engine. It converts text into realistic speech using Microsoft Edge Neural Voices.

3. import pygame

pygame is used to play audio files. In this project, it plays the generated voice.mp3 file.

4. import os

The os module is used for file handling operations like deleting temporary files.

5. async def speak(text):

This creates an asynchronous function named speak. The function accepts text as input and converts it into speech.

6. print(f"[JARVIS]: {text}")

This prints the assistant's response in the terminal. The f-string allows variables to be inserted directly into the text.

7. edge_tts.Communicate(...)

This creates a speech object using the selected AI voice. The voice 'en-US-GuyNeural' gives a natural male AI voice.

8. await communicate.save('voice.mp3')

This converts the text into speech and saves it as an MP3 audio file.

9. pygame.mixer.init()

Initializes the audio system required to play sound files.

10. pygame.mixer.music.load(...)

Loads the generated voice.mp3 file into memory.

11. pygame.mixer.music.play()

Starts playing the AI-generated speech.

12. while pygame.mixer.music.get_busy():

Keeps the program running until the audio playback finishes.

13. pygame.mixer.quit()

Closes the pygame audio system and frees resources.

14. os.remove('voice.mp3')

Deletes the temporary MP3 file after playback to keep the project folder clean.

15. asyncio.run(...)

Runs the asynchronous speak() function properly.

Overall Program Flow

Program Flow:

START → Import Libraries → Create speak() Function → Generate AI Voice → Save MP3 → Play Audio → Delete File → END